NETRA Presentation Notes

Presentation Goal

Six members, six minutes. Each person gets about 50 seconds, with 10 seconds of transition space. The story must be simple:

> Emergency reports are fragmented and duplicated. NETRA converts them into evidence-backed incidents, operational zones, explainable priorities, and human responder actions.

Do not present NETRA as a replacement for ERSS-112, a telecom network, or autonomous dispatch. Present it as an intelligence layer over existing emergency evidence.

Six-Minute Team Plan

Member 1: Problem and Context - 0:00 to 0:50

Say:

"During a disaster, the command centre receives many incomplete reports from different channels. Several reports may describe the same incident, locations may be uncertain, and the most urgent message is not always the most important rescue case. Manual triage creates duplicate work and delays the golden-hour decision. Our problem is not only message volume. It is the lack of reliable decision information from uncertain, fragmented evidence."

Transition:

"NETRA addresses this by adding an intelligence layer between emergency evidence and responder decisions."

Member 2: Proposed Solution - 0:50 to 1:40

Say:

"NETRA stands for Network-resilient Emergency Triage and Response Assistance. It accepts emergency reports, field updates, available location evidence, and simulated channel inputs. It preserves every raw event, extracts severity and vulnerability indicators, links related reports into incidents, and groups incidents into operational rescue zones. It then calculates an explainable Rescue Priority Score and recommends resources. Responders remain in control and can verify, override, rescue, or mark an incident false."

Key words:

- Evidence fusion
- Independent source counting
- Location uncertainty
- Explainable priority
- Human-in-the-loop

Member 3: Technical Approach - 1:40 to 2:35

Use the technical script below. Keep the spoken version to about 55 seconds.

Short spoken version:

"Technically, NETRA is a modular FastAPI monolith with a React and MapLibre dashboard, PostgreSQL persistence, and a database-backed asynchronous queue. The runtime path starts with an authenticated event API. The raw event is stored with source, timestamp, text, optional coordinates, accuracy, and an idempotency key. Deterministic multilingual rules run on the critical path and extract severity, vulnerability, disaster type, trapped or access indicators, and victim hints. Fusion either matches the event to a recent nearby incident using time and distance, or creates a new incident. Evidence is attached with provenance, priority is recalculated using transparent weighted factors, and operational zones are recomputed. Optional LLM enrichment runs asynchronously and cannot block the deterministic rules path. Field verification triggers recalculation again."

Transition:

"This makes every priority decision traceable from raw report to human action."

Member 4: Feasibility and Resilience - 2:35 to 3:30

Say:

"NETRA is designed to work with existing infrastructure rather than replace it. The prototype runs locally with Dockerized PostgreSQL, FastAPI, and React. If the optional AI service fails, deterministic rules continue processing. If external map tiles fail, the map falls back to a local dark grid. The dashboard exposes connectivity and AI health. For the demo, we can set the system to degraded or cellular-unavailable mode while local database processing, rules, fusion, and priority continue. This simulates loss of upstream transport; we do not claim that a browser remains connected after the laptop itself loses all network access."

Member 5: Impact and Users - 3:30 to 4:20

Say:

"The primary users are district command centres, emergency coordinators, field responders, and citizens contributing distress reports. For command centres, NETRA provides a confidence-aware incident picture. For responders, it provides priority-ranked incidents, operational zones, evidence timelines, and resource suggestions. For citizens, it improves the chance that critical indicators such as trapped people, elderly victims, children, bleeding, or rising water are visible in the triage decision. The measurable prototype outcomes are duplicate consolidation, explainable prioritization, evidence traceability, and reduced manual review effort."

Avoid saying measured speed or accuracy improvements unless a benchmark result is available.

Member 6: Demo, Validation, and Closing - 4:20 to 6:00

Recommended demo sequence:

1. Show the empty dashboard and live KPI cards.
2. Start the deterministic killer scenario with seed 42.
3. Show incoming signals becoming incidents and zones.
4. Select one incident and show priority reasons, evidence, and recommendations.

5. Use a field action such as Verify or Victim count 5.
6. Show the recommendation and priority refresh.
7. Click Net: cutout and show CELLULAR_UNAVAILABLE in the status bar.
8. Explain that local processing continues and then restore Net: normal.
9. Close with:

"NETRA does not claim to replace emergency infrastructure. It makes existing emergency evidence more usable by consolidating uncertainty, prioritizing explainably, and keeping responders in control."

If the demo is unstable, skip live injection and use the deterministic scenario. Do not spend the presentation debugging external maps or the LLM.

Your Technical Approach Notes

One-Sentence Definition

NETRA is a local-capable, modular emergency intelligence layer that converts raw, uncertain reports into evidence-linked incidents, operational zones, explainable priority levels, and human responder actions.

Architecture

Current implemented architecture:

```
React + TypeScript + MapLibre dashboard
                     |
               FastAPI API
                     |
      PostgreSQL persistence via SQLAlchemy
                     |
  DB-backed queue for optional asynchronous enrichment
```

It is a modular monolith, not a collection of microservices. The backend is split by capability: ingestion, extraction, fusion, clustering, priority, recommendation, lifecycle, auth, audit, simulation, and LLM.

Runtime Golden Path

```
POST /api/v1/events
        |
        v
Store immutable RAW Event
        |
        v
Run deterministic extraction
        |
        v
Find recent nearby active incident
or create a new incident
        |
        v
Attach Evidence with provenance
```

```
        |
        v
Update severity, victims, vulnerability,
source count, confidence, freshness
        |
        v
Calculate Rescue Priority Score
        |
        v
Recompute operational zones
        |
        v
Generate explainable resource recommendation
        |
        v
Queue optional LLM enrichment
        |
        v
Field update triggers priority, recommendation,
and zone recalculation
```

Ingestion

The ingestion router accepts a typed event with:

- `source_type`: SMS, ERSS, ELS, WHATSAPP, FIELD, MANUAL, or SIMULATED
- `source_timestamp`
- optional text
- optional latitude, longitude, and accuracy
- pseudonymized source identifier
- idempotency key

The event is stored before processing. If the same idempotency key is submitted again, the existing event is returned instead of creating a duplicate.

Important accuracy statement:

> The prototype has a common event contract and simulator/API inputs. It does not yet contain live SMS, ERSS, WhatsApp, or official-alert adapters.

Deterministic Extraction

The L1 rules path is always available and runs synchronously. It supports English, Hindi, and romanized Hinglish terms. It extracts:

- Severity: critical, high, medium, unknown
- Vulnerabilities: elderly, child, mobility issue, pregnant
- Water rising
- Trapped status
- Access issue

- Disaster type: flood, earthquake, cyclone
- Safe or rescued language
- Suspected fake/test messages
- Victim count hints

Every extracted attribute includes a rule/model version, confidence, and matching source terms. This makes the result auditable.

Do not say "AI intent filtering" as if the system depends on an opaque model. Say:

> Deterministic multilingual safety extraction is on the critical path; optional LLM enrichment is asynchronous.

Fusion and Deduplication

Fusion does not count every message as a new victim or a new incident.

For a located event, the prototype searches active incidents within a recent time window and an adaptive distance radius. The radius uses location accuracy and is capped. The nearest candidate is selected. Events without usable coordinates cannot be spatially matched and may create separate incidents.

Evidence remains linked to the incident. The incident tracks:

- Evidence count
- Independent source count
- Last evidence time
- Confidence
- Vulnerability indicators
- Location confidence

Important distinction:

> Duplicate evidence can increase corroboration and confidence, but independent source count is tracked separately so 84 messages are not treated as 84 victims.

Location and Clustering

Current prototype behavior:

- Coordinates are stored as scalar latitude and longitude fields.
- Location accuracy is stored separately.
- Python haversine distance is used for event-to-incident matching.
- Operational zones are recomputed by the clustering service.
- MapLibre renders incidents, heat points, and zone circles.

Do not claim active PostGIS geometry or GIST indexes. The project uses a PostGIS Docker image, but the current ORM schema has not migrated to geometry columns.

Correct phrase:

> Coordinate-aware, uncertainty-aware clustering with a PostGIS-ready deployment path.

Avoid:

> Fixed 50 metre PostGIS DBSCAN is already implemented.

Rescue Priority Score

The score is explainable and versioned. Current weighted factors are:

- Severity: 30%
- Vulnerability: 20%
- Victim estimate: 15%
- Freshness: 15%
- Location confidence: 10%
- Access risk: 10%

Independent corroboration applies a bounded boost. Priority levels are P1, P2, P3, and P4. Hysteresis prevents a priority from dropping too quickly when the evidence temporarily becomes less fresh.

Explain it like this:

> Priority is not message count. It combines the seriousness of the situation, who is vulnerable, how many victims are indicated, how recent the evidence is, how reliable the location is, and whether access is blocked. Every score returns reasons that the operator can inspect.

Recommendations

Recommendations are deterministic and explainable. Examples:

- Rescue boats for rising water, trapped reports, or high severity
- Medical team for high/critical severity or vulnerable people
- Ambulance standby for pregnancy indicators
- Road clearance for access issues
- Shelter information for lower-risk informational cases

These are suggestions, not dispatch commands. The operator or commander decides what to accept.

Human-in-the-Loop Lifecycle

Field updates include Verify, Victim count, Access, Medical, Rescued, False, and Note. Every update is stored as a human decision and written to the audit log.

A field update can:

1. Change incident state or attributes
2. Increase confidence after verification
3. Recalculate priority
4. Replace the recommendation
5. Recompute operational zones
6. Remove a rescued or false incident from the active queue

This is one of NETRA's strongest technical points because the system is not pretending the model is always correct.

Async LLM Path

After an event with text is processed, an LLM enrichment job may be queued. The queue is database-backed and processed by a worker thread. LLM failure is recorded and does not block the deterministic event-to-incident path.

Current AI maturity:

- L1 deterministic rules: implemented
- L2 local classifier: not implemented in the current prototype
- L3 async LLM gateway: implemented as optional enrichment/fallback path

Say this honestly if asked. Honest scope is stronger than claiming an unfinished layer.

Network Cutout Demonstration

Use this exact explanation:

> We are not claiming that the entire cellular network is still usable after a physical blackout. We are simulating loss of upstream/cellular transport. The local NETRA database, deterministic rules, fusion, priority, and existing operational state continue. External map tiles and optional LLM services may degrade independently.

Demo steps:

1. Start the killer scenario.
2. Wait until incidents and zones appear.
3. Click `Net: cutout`.
4. Show `CELLULAR_UNAVAILABLE` in the status bar.
5. Inject or process local data if the backend remains reachable.
6. Explain the local-first path.
7. Click `Net: normal`.

What the Dashboard Proves

The dashboard is not only a map. Point to these regions:

- KPI cards: volume, incidents, open response, P1 count, signal mix
- Left queue: incident prioritization and selection
- Map: location, heat, operational zones, live counts
- Right panel: score, reasons, resources, field actions, evidence timeline
- Status bar: connectivity and LLM health
- Signal stream: recent incoming evidence

What It Does Not Prove

Do not claim that the dashboard proves:

- Real government ERSS integration
- Physical SMS delivery or telecom resilience
- Active PostGIS geometry queries
- Autonomous dispatch
- A completed L2 classifier
- Measured improvement in triage time or accuracy unless a benchmark result is shown

Judge Questions and Answers

Why not just use a normal GIS dashboard?

A GIS dashboard shows locations. NETRA adds evidence fusion, independent-source counting, confidence, explainable priority, freshness, and recommendations. The differentiator is the decision layer between incoming reports and the map.

Why is independent-source counting important?

Many messages can describe the same incident. Counting messages as victims creates false urgency. NETRA preserves all evidence but tracks independent source identifiers separately and applies a bounded corroboration boost.

What happens if the LLM fails?

The LLM is off the critical path. Deterministic multilingual rules still extract core safety indicators, and the event can still be fused, prioritized, and shown to a responder. The dashboard exposes LLM health.

What happens if location is missing or inaccurate?

The event is still stored. Location confidence is reduced. Spatial matching is only used when coordinates are available, and the UI can show unlocated incidents rather than inventing a precise point.

Is it autonomous dispatch?

No. NETRA recommends resources and prioritizes incidents. Human responders verify and decide. This is deliberate because life-critical action must remain under human control.

Is PostGIS implemented?

The deployment uses a PostGIS image, but the current prototype stores scalar coordinates and uses Python haversine matching. Geometry columns and spatial indexes are a planned production migration.

How does it scale?

The prototype uses a modular monolith and a database-backed queue to stay simple and deterministic. The capability boundaries allow future extraction of ingestion, NLP, geospatial processing, or queue workers if load requires it. Do not claim production-scale throughput without benchmark evidence.

What does network-resilient mean?

It means the local decision path degrades gracefully when external connectivity, tiles, or LLM services fail. It does not mean NETRA can receive new reports from a physically disconnected device without any available transport.

What is the core innovation?

The Rescue Priority Score plus evidence fusion: a transparent way to convert uncertain, duplicated, multi-source reports into a prioritized rescue picture while preserving provenance and human control.

Final 20-Second Closing

"NETRA turns fragmented emergency evidence into a decision-ready rescue picture. It preserves the raw report, explains how the incident was scored, counts independent evidence correctly, recommends resources, and lets responders verify or override the decision. Our prototype is local-capable, deterministic for demonstration, and designed to integrate above existing emergency systems rather than replace them."

Rehearsal Checklist

- Keep the whole team under six minutes.
- Each person speaks from one message, not every feature.
- Technical speaker uses the golden path once, clearly.
- Do not say PostGIS geometry, fixed 50m DBSCAN, real adapters, or autonomous dispatch unless explicitly marked as future work.
- Start the backend before the frontend.
- Start the scenario before the live demo.
- Keep a scripted killer-scenario fallback ready.
- If the map tiles fail, explain the local dark-grid fallback and continue.
- If LLM health is degraded, use it as the resilience demonstration.
- End with the human-in-the-loop point.

server staring

1. Start backend
   cd /mnt/newvolume/SIH-Hackathon/NETRAv1/backend && setsid nohup .venv/bin/python -m uvicorn app.main:app --host 0.0.0.0 --port 8001 < /dev/null > /tmp/netra-8001.log 2>&1 & disown
   Check: curl -s localhost:8001/healthz → should say "database":"healthy".
2. Start frontend
   cd /mnt/newvolume/SIH-Hackathon/NETRAv1/frontend && npm run dev -- --host 0.0.0.0 &
3. Start tunnel (needs internet)
   /tmp/opencode/cloudflared tunnel --url http://localhost:5173
   Copy the https://....trycloudflare.com URL it prints. If rebooted, re-download first:
   curl -sL -o /tmp/opencode/cloudflared https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64 && chmod +x /tmp/opencode/cloudflared
4. Pre-demo setup (~15 min before judges)

- Open http://localhost:5173/sim → login commander / commander123
- Click Killer scenario (LLM enrichment needs ~10 min to fully drain — start early)
- Click ← open dashboard — clean monitoring view

5. Interactive segment (judges become victims)

- Projector: open /victim → shows QR
- Judges scan → tap Allow GPS → pick preset → SEND SOS
- Dashboard shows incidents/zones forming live + their report scrolls in the NEWS WIRE

6. Adversarial beats (from /sim)

- Fake SOS / duplicates → watch confidence drop
- LLM: kill → status flips DEGRADED, rules fallback keeps working
- Net: cutout → judges' WHATSAPP/ERSS fail with "uplink unavailable", the SMS one gets through

7. Live proof — open http://localhost:5173/logs → login admin / admin123 → watch the audit trail stream
8. After demo — http://localhost:5173/sim → Reset demo state
   Fallbacks: no internet → skip tunnel, use http://192.168.1.133:5173/victim on the same Wi-Fi (GPS needs HTTPS, so phones will fall back to simulated spots). No Wi-Fi at all → run killer scenario only from /sim.